

Fair Repetitive Interval Scheduling

Yuval Itzhaki

Claude

Abstract

This note presents the formalization of the paper of Heeger, Hermelin, Itzhaki, Molter and Shabtay [1]: n clients each submit one just-in-time job on every one of m days, and a schedule is *fair* if every client is served on at least k of the days. The paper gives a complete dichotomy of the problem's complexity in the fairness parameter k , settles the two natural restrictions to day-independent processing times and day-independent due dates, and determines the complexity with respect to the treewidth of the daily conflict structure and the number of clients. The development proves the scheduling results and supporting algorithmic statements in Lean. The construction gives inactive clients disjoint slots inside the dummy client's interval, preserving the conflict graph needed for the treewidth bound. A reduction from [2,3]-BOUNDED 3-SAT establishes the required hardness of MULTICOLOURED INDEPENDENT SET on regular instances. This note presents the statements in the notation of [1], with links to their Lean counterparts.

1 Introduction

Definition 1 ($1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$, EQUIJIT). n clients and m days. Client j 's job on day i has a processing time $p_{i,j} \geq 1$ and a due date $d_{i,j} \geq p_{i,j}$, and occupies exactly the interval $(d_{i,j} - p_{i,j}, d_{i,j}]$. Two jobs of the same day *conflict* if their intervals intersect. One machine is available on each day; a *schedule* selects, for every day, a set of clients whose jobs that day are pairwise non-conflicting (the rest are rejected that day). Writing $Z_{i,j}$ for the indicator that client j is served on day i , a schedule is *k-fair* if $\sum_i Z_{i,j} \geq k$ for every client j . Given k , is there a k -fair schedule?

Running times below are stated on the word RAM: a program is a fixed list of instructions operating on words of w bits, and its running time is the number of instructions it executes on an input word, under the word-length and workspace conditions stated in the linked concepts.

[1] gives a complete answer in the fairness parameter alone:

Theorem 1. For $0 \leq k \leq m$, EQUIJIT is solvable in polynomial time when $k \in \{0, m-1, m\}$. It is NP-hard for every fixed pair (m, k) with $0 < k < m-1$.

and then studies two restrictions — *day-independent processing times* ($p_{i,j} = p_j$) and *day-independent due dates* ($d_{i,j} = d_j$) — and two structural parameters of the *overall conflict graph* G : the graph on the clients in which two are adjacent exactly when some day's jobs of theirs conflict.

2 Preliminaries

A day's conflict graph is an interval graph, and a schedule is feasible on a day exactly when its set of clients is independent in that day's graph. The *overall conflict graph* G is the union, over all days, of the daily conflict graphs; its *treewidth* τ is the structural parameter of Section 5. A tree decomposition of a graph is a tree of bags of vertices, every vertex and every edge in some bag, the bags containing a fixed vertex forming a connected subtree; its width is the largest bag size minus one, and the treewidth of a graph is the least width of a tree decomposition of it.

Bodlaender’s algorithm finds one of minimum width, if it exists within a given bound, in time $2^{O(\tau^3)} \cdot \text{poly}$. An algorithm is *fixed-parameter tractable* (FPT) for a parameter κ if it runs in $f(\kappa) \cdot n^{O(1)}$ time for some computable f .

3 The Fairness Parameter

Theorem 7. *EQUIJIT is NP-hard already at $k = 1$, $m = 3$, and with every job’s processing time equal.*

The reduction is from [2,3]-BOUNDED 3-SAT: given a formula with ℓ two-literal and q three-literal clauses over n variables, every processing time is 2, so two jobs of a day conflict exactly when their due dates differ by at most one, and the whole construction places due dates on a line, three times over — three *dummy* clients, two clients x_v^1, x_v^0 per variable, and one client per literal occurrence, with due dates chosen so that a 1-fair schedule of the three days exists exactly when the formula is satisfiable.

Corollary 8. *EQUIJIT is NP-hard for every $m \geq 3$ and $0 < k < m - 1$.*

Two reductions lift Theorem 7 to every such pair: adding one conflict-free day raises both m and the attainable fairness by one, taking (m, k) to $(m + 1, k + 1)$; and adding one day together with one new client that conflicts with every job of every old day, and whose own job occupies the new day alone, forces the new client onto the new day in any 1-fair schedule, taking $(m, 1)$ to $(m + 1, 1)$. Chaining these from $(3, 1)$ reaches every (m, k) with $m \geq 3$ and $0 < k < m - 1$.

Theorem 9. *EQUIJIT is solvable in polynomial time when $k = m - 1$.*

A variable $x_{i,j}$ per day and client; a *conflict clause* $(\neg x_{i,j_1} \vee \neg x_{i,j_2})$ for every conflicting pair on a day, and a *validation clause* $(x_{i_1,j} \vee x_{i_2,j})$ for every client and every two days — forbidding two rejections — give a 2-CNF formula satisfiable exactly when a $(m - 1)$ -fair schedule exists. The formalization uses its verified polynomial-time 2-SAT algorithm.

Theorem 1 follows from Corollary 8 and Theorem 9, together with the trivial cases $k = 0$ and $k = m$.

4 Day-Independent Processing Times and Due Dates

Theorem 2 (Hardness). *DAYINDEPP-EQUIJIT is NP-hard.*

Theorem 7’s reduction already produces instances of equal — hence day-independent — processing times, so Corollary 8’s lift carries this hardness to every $m \geq 3$, $0 < k < m - 1$.

Theorem 10. *With unit processing times, two jobs of a day conflict exactly when their due dates agree. An instance with $k \leq m$ admits a k -fair schedule if and only if the bipartite graph with a job vertex $v_{i,j}$ per day and client, a due-date vertex $u_{i,t}$ per day and distinct due date, and $m - k$ rejection vertices $w_{1,j}, \dots, w_{m-k,j}$ per client — joining $v_{i,j}$ to $u_{i,d_{i,j}}$ and to every rejection vertex of client j — has a matching saturating every job vertex.*

Theorem 2 (Tractability). *UNIT-EQUIJIT is solved in polynomial time, by Theorem 10 and a maximum bipartite matching algorithm.*

Theorem 11. *DAYINDEPD-EQUIJIT is NP-hard.*

Reading an instance of $R \parallel \sum_j Z_j$ — unrelated machines, maximizing the just-in-time jobs — as one of DAYINDEPD-EQUIJIT with jobs as clients and machines as days gives an instance at $k = 1$ that is a yes-instance exactly when every job of the source can be scheduled just in time, which is NP-hard [2].

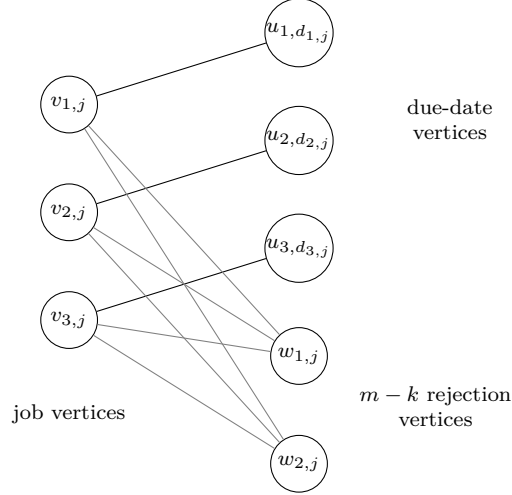


Figure 1: The bipartite graph of Theorem 10 for one client j , $m = 3$, $k = 1$: matching a job vertex to its due-date vertex records that the client is served that day, matching it to a rejection vertex records that it is not, and only $m - k$ rejections are available.

Theorem 12. *With day-independent due dates and a constant number m of days, EQUIJIT is solved in polynomial time.*

Ordering the clients by due date, a dynamic program carries, for each day, the time at which its machine next frees up; serving a client on a set of days needs each of those machines free early enough, and leaves each free from the client's due date onwards. Since later clients have due dates at least as large, this state is exactly what the decision about earlier clients needs to leave behind, and for constant m there are polynomially many states.

Theorem 13. *With both due dates and processing times day-independent, every day has the same conflict graph G , and DAYINDEP-EQUIJIT is solvable in polynomial time: a k -fair schedule exists exactly when $k \cdot \chi(G) \leq m$, where $\chi(G)$ is the chromatic number of G .*

A $\chi(G)$ -colouring partitions the clients into $\chi(G)$ independent sets, scheduled in rotation; conversely a clique of G needs one day per client per repetition, and since G is an interval graph its chromatic number equals its clique number, so the two bounds coincide.

5 Complexity of Structural Parameters

Theorem 4 (Bounded-Treewidth Hardness). *EQUIJIT is NP-hard on instances whose overall conflict graph has treewidth at most 6, and hence under any fixed treewidth upper bound of at least 6.*

The reduction is in two steps: a per-client generalization at treewidth 4 (Lemma 14), then a uniform-parameter reduction raising the treewidth by at most 2 (Lemma 15).

5.1 Hardness at Bounded Treewidth

MULTICOLOURED INDEPENDENT SET: given an ℓ -partite graph $G = (V_1 \cup \dots \cup V_\ell, E)$, is there a choice of one vertex per class with no two chosen vertices adjacent? It is NP-hard.

Regular Source Instances. Lemma 14's reduction uses G in *normal form*: r -regular with $r \geq 1$, every class of $n \geq 4$ vertices, and $|E|$ even. The formalization proves MULTICOLOURED INDEPENDENT SET NP-hard on these instances directly,

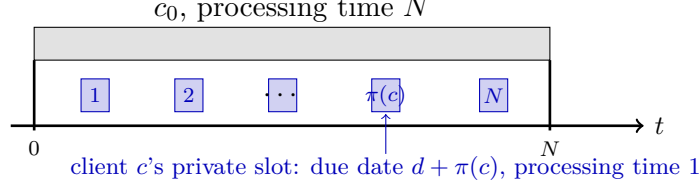


Figure 2: Blocking on one day: c_0 's job occupies $(0, N]$, and every inactive client's job is its own private unit slot at offset $\pi(c)$ — conflicting with c_0 but with no other blocked client.

by a reduction from [2,3]-BOUNDED 3-SAT [3]: the occurrence graph of a formula (one vertex per clause position) is made regular by five uniform *ports* per position, each a seven-vertex gadget whose independence number is fixed whether or not its port is used, and the resulting graph is then replaced by a disjoint union of copies of itself, with the colour classes defined by the construction, which fixes the parity of $|E|$ without touching r -regularity or the existence of a multicoloured independent set.

Lemma 14. $1 \mid k_j, \text{rep} \mid \min_j \sum_i Z_{i,j} - \text{EQUIJIT}$ with one fairness parameter k_j per client, WEIGHTED-EQUIJIT — is NP-hard even when the overall conflict graph has treewidth at most 4.

From a normal-form MULTICOLOURED INDEPENDENT SET instance G : $\ell(n+1) + |E|$ days — for every colour n *vertex days* and one *validation day*, and one *edge day* per edge — and clients: a *vertex client* c_v ($k_{c_v} = 1$) per vertex, a *selection client* c_i ($k_{c_i} = 1$) per colour, two *edge clients* $c_{v,u}, c_{u,v}$ ($k = 1$) per edge, two *interaction clients* c^+, c^- ($k^+ = k^- = |E|/2$), and a *dummy client* c_0 with k_0 equal to the number of days. On every day, only a handful of clients take part in that day's gadget, and c_0 's job — the longest of the day — *blocks* every other client not named in the gadget, forcing it to be rejected that day.

Blocking Inactive Jobs. Giving all inactive clients the same interval would make them pairwise adjacent in the conflict graph. To preserve the treewidth bound, each inactive client instead occupies a separate unit-length slot inside c_0 's interval: numbering the clients other than c_0 by a bijection π with $\{1, \dots, N\}$, c_0 's job has processing time N and due date $r + N$ (say, on a vertex day), and a blocked client c 's job is the single time unit at offset $\pi(c)$, due date $r + \pi(c)$. Every blocked client still conflicts with c_0 — its slot lies inside c_0 's interval — but no two blocked clients conflict with each other, since their slots are disjoint. Figure 2 shows the resulting gadget on one day. The formalization proves the stated tree decomposition for this construction.

For every colour, the vertex and validation days form a *vertex-selection* gadget compelling exactly one vertex client per colour onto its validation day; the edge days form an *incidence-checking* gadget, in which c^+ and c^- can together be satisfied exactly when the selected vertices are pairwise non-adjacent. The resulting conflict graph has the tree decomposition of Figure 3: a root bag $\{c_0, c^+, c^-\}$, one child bag per colour adding c_i , one grandchild per vertex of that colour adding c_v , and one great-grandchild per neighbour u of v replacing c_i with $c_{v,u}$. Thus every bag has at most five clients, giving width at most four.

Lemma 15. $1 \mid k_j, \text{rep} \mid \min_j \sum_i Z_{i,j}$ reduces in polynomial time to $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$, raising the treewidth of the overall conflict graph by at most 2.

Given I with n clients, m days and parameters $k_j \leq m$: append another m days and two clients c^-, c^+ whose identical job, on every new day, occupies a stretch later than every due date

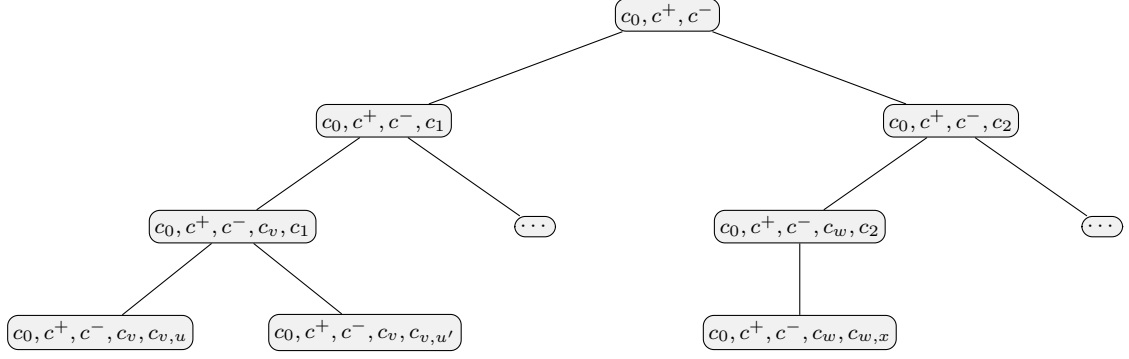


Figure 3: The shape of Lemma 14's tree decomposition, for two colours.

of I — so it conflicts with nothing of I — and ask for the uniform parameter m . Each of c^-, c^+ requires m executions. They conflict on every day, so together occupy all $2m$ days, with exactly one of them served each day. Each original client j is blocked on k_j of the additional days and can run on the other $m - k_j$ days. It can therefore reach the uniform requirement m exactly when it can be served on at least k_j original days. Two clients add at most two to the treewidth.

Combining Lemmas 14 and 15 gives Theorem 4: $4 + 2 = 6$.

5.2 Finding Nice Tree Decompositions

The dynamic programs below consume *nice* tree decompositions, whose nodes are leaves, *introduce* nodes that add one vertex to the bag of their child, *forget* nodes that remove one, and *join* nodes with two children of equal bag. For the word RAM, a graph on n vertices is the word n followed by its adjacency matrix row by row, and a nice decomposition is a word of N records, one per node, listing its kind, its vertex and its second child, each node after its children; the bags are determined from the leaves upward.

Theorem (Bodlaender and Kloks). *Given a graph and a nice tree decomposition of width at most ℓ , one can decide whether the treewidth is at most k and, if so, find a nice decomposition of width at most k , in time $2^{O(\ell^3)} \cdot \text{poly}$.*

Theorem (Bodlaender). *Given a graph and k , one can decide whether the treewidth is at most k and, if so, find a nice decomposition of width at most k , in time $2^{O(k^3)} \cdot \text{poly}$.*

Bodlaender's theorem is formalized with an algorithm that is done slightly differently than in the paper: the exact dynamic program of Bodlaender and Kloks over *characteristics* of partial decompositions, applied vertex by vertex. A decomposition of the first i vertices is extended to one of the first $i + 1$ by inserting the new vertex into every bag, which raises the width by one, and the improvement step of the previous theorem brings the width back down. Its running time is proved on a verified virtual machine for a functional language, whose runs are transferred to the word RAM, and is polynomial in the length of the word times $2^{O(k^3)}$. The statement used for Theorem 4 is its case for the overall conflict graph of an instance.

5.3 Fixed-Parameter Tractability

Theorem 4 (Days and Treewidth). *EQUIJIT is fixed-parameter tractable with respect to $m + \tau$.*

A dynamic program over a nice tree decomposition records the schedules on each bag that extend to feasible, fair schedules on the clients in its subtree. A state specifies, for each day, which clients of the bag are served. The transitions combine these restrictions at introduce, forget and join nodes. The verified implementation establishes fixed-parameter tractability; no sharper running-time bound is asserted here.

Theorem 4 (Number of Clients). *EQUIJIT is fixed-parameter tractable with respect to the number n of clients.*

The reduction groups days by their conflict relation. The formal construction reserves 2^{n^2} relation types and 2^n subsets of clients, giving exactly $2^{n^2+n} + n$ variables, including one slack variable per client, and $2^{n^2} + n$ constraints. The constraint matrix depends only on n ; the right-hand sides record counts of day types and the allowed number of rejections.

The source uses fixed-dimension integer programming [4]. The formalization instead proves an algorithm for this particular family. It enumerates certificates in a finite search space whose size depends only on n , reconstructs candidate solutions from each certificate and the right-hand sides, and checks the constraints. The normalization and certificate-completeness lemmas justify this search. This establishes fixed-parameter tractability without enumerating every assignment up to the right-hand sides. The scheduling decision program separately handles short inputs by enumeration.

6 Formalization Notes

Running-time bounds are verified for word RAM programs compiled through the archive’s IMP+ compiler ([1ax-808846](#)). The submission includes proofs of its 2-SAT and bipartite matching algorithms, and of the algorithm of Bodlaender and Kloks for finding nice tree decompositions of small width. It uses Tovey’s bounded-occurrence satisfiability theorem ([1ax-345332](#)), interval scheduling with eligible machine sets ([1ax-888481](#)), and the Hitting Set hardness of just-in-time scheduling in flexible flow shops ([1ax-496464](#)). The hardness arguments also use the archive’s Cook–Levin theorem and CNF encoding ([1ax-429075](#)) and the equivalence of word RAM and Turing-machine polynomial time ([1ax-759944](#)). The concept pages and proof network specify the hypotheses and dependencies of each result.

7 Discussion

The formalization adjusts the placement of inactive jobs to preserve the conflict graph required by the treewidth argument. A dummy interval blocks the inactive jobs, which occupy separate, disjoint slots within it. The development also supplies the NP-hardness proof for the regular MULTICOLOURED INDEPENDENT SET instances used by the construction. The concept pages describe these details and the resulting tree decomposition.

References

- [1] K. Heeger, D. Hermelin, Y. Itzhaki, H. Molter and D. Shabtay. Fair Repetitive Interval Scheduling. *Algorithmica*, 2025. DOI: [10.1007/s00453-025-01322-y](#)
- [2] S. C. Sung and M. Vlach. Maximizing Weighted Number of Just-in-Time Jobs on Unrelated Parallel Machines. *Journal of Scheduling* 8(5):453–460, 2005. DOI: [10.1007/s10951-005-2860-z](#)
- [3] C. A. Tovey. A Simplified NP-Complete Satisfiability Problem. *Discrete Applied Mathematics* 8(1):85–89, 1984. DOI: [10.1016/0166-218X\(84\)90081-7](#)
- [4] H. W. Lenstra, Jr. Integer Programming with a Fixed Number of Variables. *Mathematics of Operations Research* 8(4):538–548, 1983. DOI: [10.1287/moor.8.4.538](#)